УНИВЕРСИТЕТ ПУЧОН В ГОРОДЕ ТАШКЕНТЕ

Кафедра e-Business

Программная инженерия

ПРОЕКТНАЯ РАБОТА

Глава 3. Проектирование

Предзаказ обеда в университетской столовой

Команда 1

A — Шашкина Виктория: Product Owner и бизнес-аналитик

B — Алиев Салим: системный аналитик и UX-дизайнер

C — Фаткулбаянов Руслан: менеджер проекта и QA-инженер

Преподаватель: Ботиров С. Р.

Репозиторий команды: https://github.com/Zombik333/team1-predzakaz-obeda

Прототип в Figma: https://www.figma.com/proto/rmpJx3sDMnKO9yX4EyJlqL/?node-id=1-2&starting-point-node-id=1%3A2&scaling=scale-down&content-scaling=fixed&page-id=0%3A1

Версия 1.0 · Ташкент, 2026

Содержание

Введение к главе 3 3
3.1. Бизнес-процессы 3
3.1.1. Процесс «как есть» 3
3.1.2. Процесс «как будет» 5
3.1.3. Что изменилось в процессе 7
3.1.4. Матрица RACI 7
3.2. UML-модели 8
3.2.1. Диаграмма вариантов использования 8
3.2.2. Диаграммы деятельности 11
3.2.3. Диаграмма последовательности: оплата заказа 13
3.2.4. Диаграмма классов предметной области 14
3.3. Модель данных 15
3.4. Архитектура 18
3.4.1. Архитектурный стиль 18
3.4.2. Уровни и модули 18
3.4.3. Интеграции и риски 21
3.5. Прототип интерфейса 23
3.5.1. Экраны и переходы 23
3.5.2. Проверка прототипа на 5 участниках 24
3.5.3. Самопроверка по 10 эвристикам Нильсена 25
3.6. Выбор платформы и стоимость владения 27
3.7. Трассировка: дополнение к матрице 2.7 29
Выводы по главе 3 30
Приложение Г. Описание таблиц базы данных 31
Приложение Д. Сценарий проверки прототипа 34
История изменений 35

Обновите поле содержания в Word: правый клик → «Обновить поле».

Введение к главе 3

Глава 3 показывает, как сервис предзаказа выполняет требования главы 2. Мы строим модели в таком порядке: бизнес-процесс, варианты использования, данные, архитектура, интерфейс. Каждая модель ссылается на истории US, требования FR и NFR и ограничения C из главы 2.

Цель продукта (BG-0). За 3 месяца после запуска студент получает обед в большую перемену без очереди, а столовая ведёт учёт каждого онлайн-заказа. Задачи BG-1…BG-4 работают на эту цель. В главе 3 каждое решение мы связываем с одной из этих задач.

Источники. Глава 2, версия 1.1: истории US-01…US-20, требования FR-01…FR-27 и NFR-01…NFR-10, ограничения C-01…C-06, допущения A-01…A-07, глоссарий. Данные интервью П1–П5, опроса В1–В10 и замеров очереди. Нотации: BPMN 2.0 и UML 2.5. СУБД: PostgreSQL 16.

Файлы моделей. Все диаграммы лежат в репозитории команды, в папке /models, в форматах SVG и PNG. Схема базы данных также записана в формате DBML (файл db_schema.dbml) и в виде SQL-скрипта (файл schema.sql). Экраны прототипа лежат в папке /prototype.

Исправления по замечаниям к этапу 2. Мы выполнили все замечания преподавателя.

- Раздел 2.1.2 теперь называется «Бизнес-цель и задачи»: одна цель BG-0 и 4 задачи BG-1…BG-4. Номера задач не изменились, поэтому матрица 2.7 осталась верной.
- В презентации этапа 2 есть отдельный слайд для функциональных требований (27 FR по ролям) и отдельный слайд для нефункциональных требований (10 NFR).
- На слайдах нет пустых зон. Яркий рыжий цвет мы заменили на спокойные цвета: тёмно-синий, песочный и приглушённый синий. Эти цвета мы используем и в главе 3.

3.1. Бизнес-процессы

Автор раздела: Шашкина Виктория (А)

В разделе 3.1 мы сравниваем процесс получения обеда сейчас («как есть») и с сервисом («как будет»). Модели построены в нотации BPMN 2.0. Каждая модель имеет пулы и дорожки, задачи с глаголом, подписи всех ветвей шлюзов, начальные и конечные события.

3.1.1. Процесс «как есть»

Сначала мы описываем, как студент получает обед сейчас. Модель построена по интервью П1–П5, опросу и замерам очереди из главы 2. Рисунок 3.1 показывает этот процесс. Номера в кругах отмечают проблемы процесса.

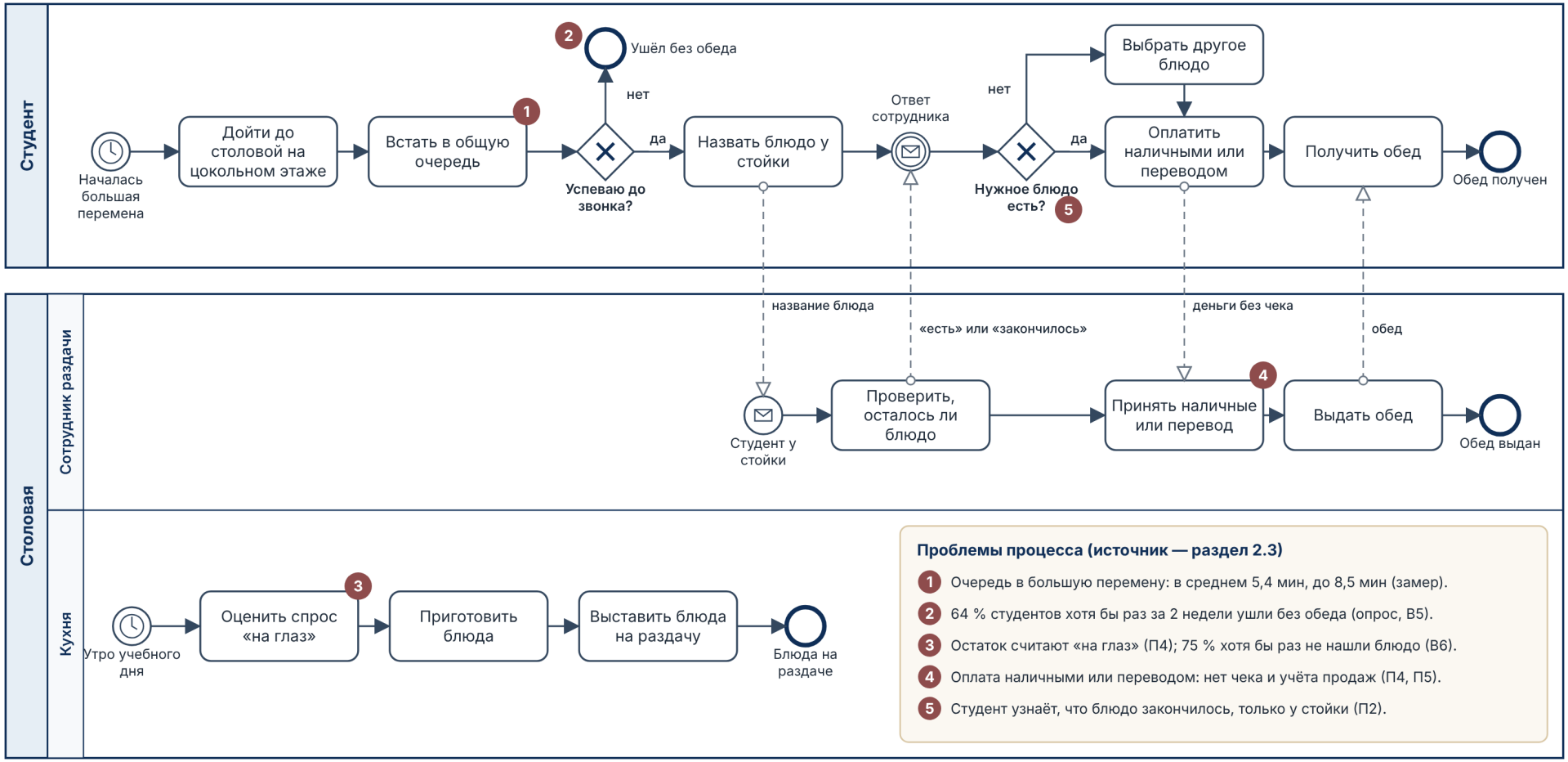


Рисунок 3.1 — Процесс получения обеда «как есть» (BPMN 2.0)

В процессе два участника: пул «Студент» и пул «Столовая» с дорожками «Сотрудник раздачи» и «Кухня». Студент начинает процесс по таймеру «Началась большая перемена». Шлюз «Успеваю до звонка?» имеет две ветви: «да» ведёт к стойке, «нет» заканчивает процесс событием «Ушёл без обеда». Шлюз «Нужное блюдо есть?» показывает, что студент узнаёт о наличии блюда только у стойки. Кухня оценивает спрос «на глаз» и не знает, сколько заказов будет. Оплата идёт наличными или переводом на карту сотрудника, без чека.

Вывод. Процесс «как есть» имеет 5 проблем: общая очередь (в среднем 5,4 минуты, максимум 8,5), уход без обеда (64 % студентов, вопрос В5), нет нужного блюда (75 %, В6), оплата без чека (П4, П5) и неизвестный остаток (П2). Процесс «как будет» должен убрать эти проблемы.

3.1.2. Процесс «как будет»

Процесс «как будет» мы разделили на две части, чтобы модель было удобно читать. Часть 1 описывает заказ и оплату (рисунок 3.2). Часть 2 описывает сборку и выдачу заказа (рисунок 3.3). Части соединяет связующее событие «К части 2».

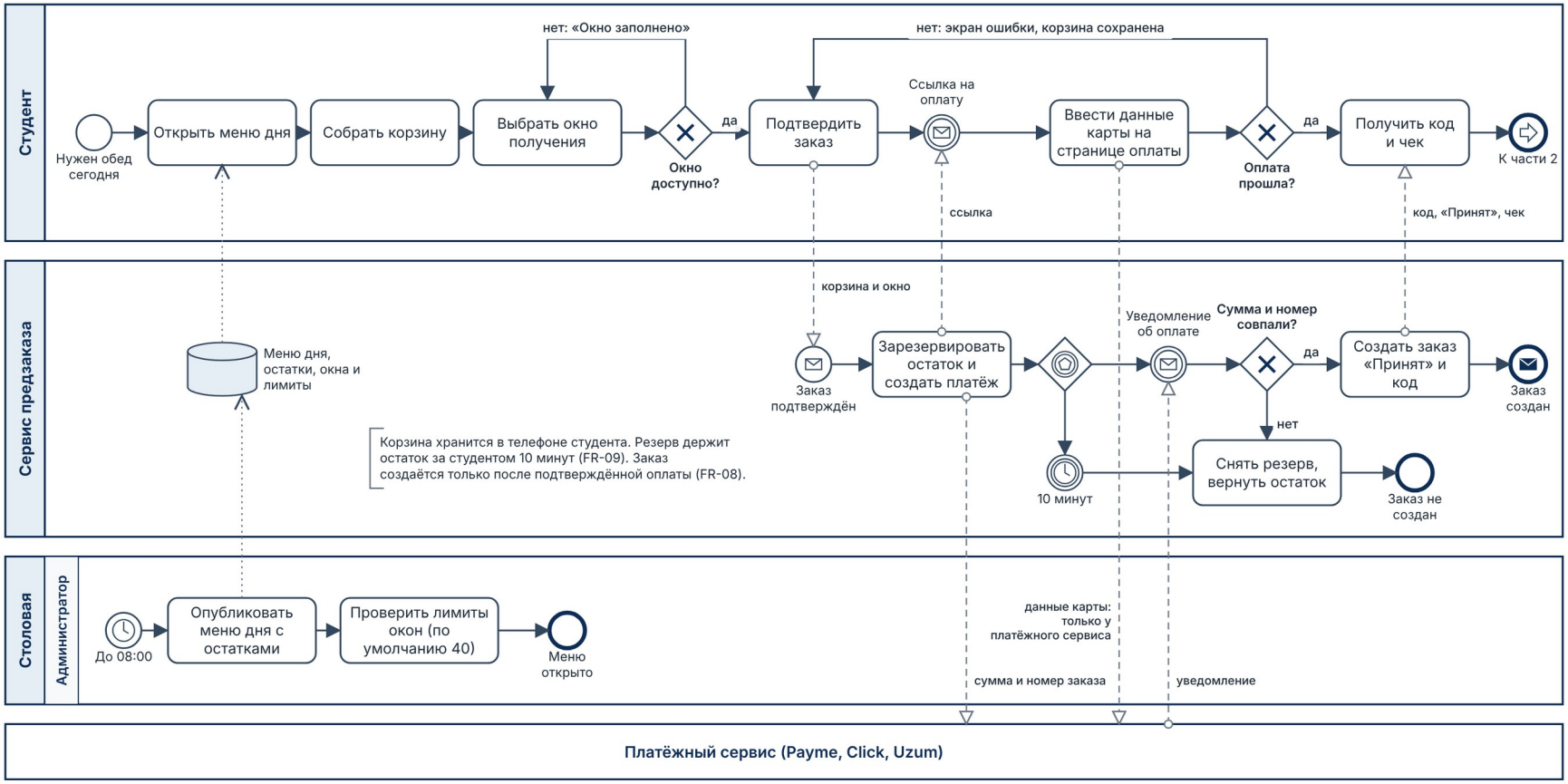


Рисунок 3.2 — Процесс «как будет», часть 1: заказ и оплата (BPMN 2.0)

В части 1 четыре пула: «Студент», «Сервис предзаказа», «Столовая» (дорожка «Администратор») и свёрнутый пул «Платёжный сервис». Администратор публикует меню дня с остатками до 08:00 (таймерное событие) и проверяет лимиты окон. Если окно заполнено, шлюз возвращает студента к выбору окна. Сервис резервирует остаток и создаёт платёж. Шлюз на основе событий ждёт одно из двух событий: уведомление об оплате или таймер «10 минут». Если таймер срабатывает первым, сервис снимает резерв и возвращает остаток (FR-09). Заказ создаётся только после проверки суммы и номера платежа (FR-08). Данные карты студент вводит только на странице платёжного сервиса (NFR-05, C-02).

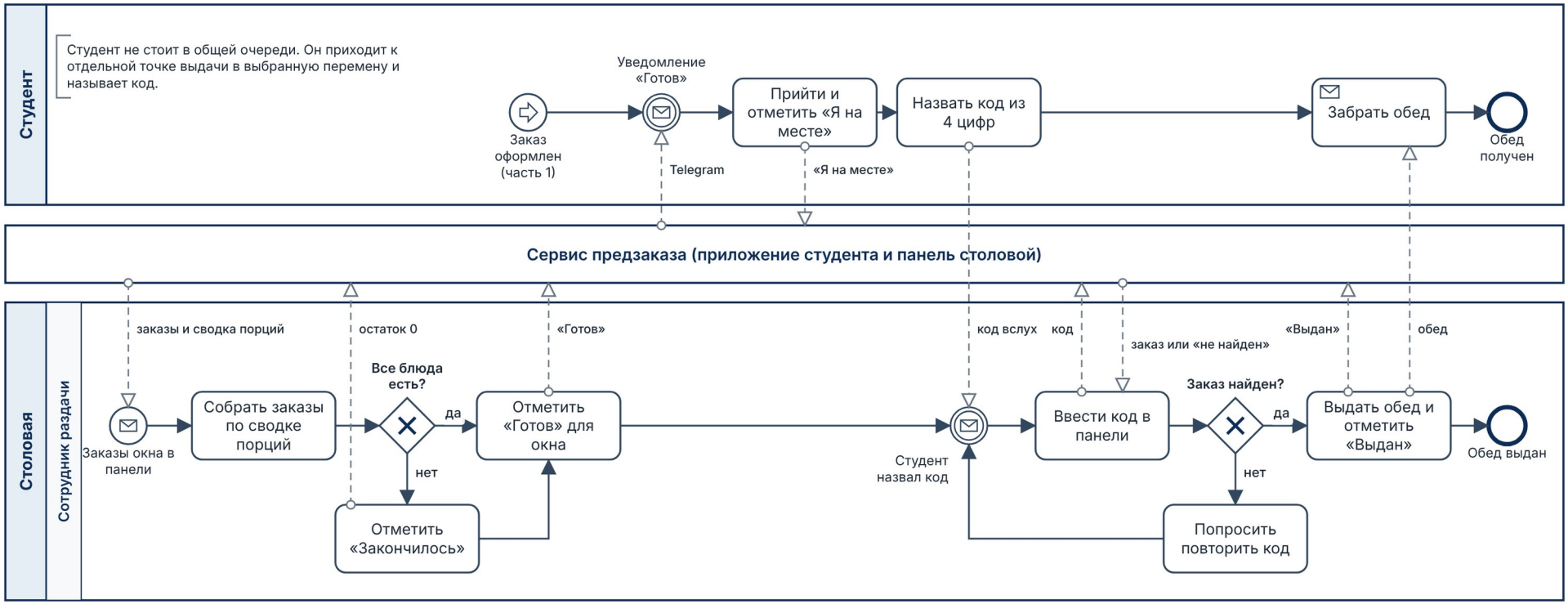


Рисунок 3.3 — Процесс «как будет», часть 2: сборка и выдача (BPMN 2.0)

В части 2 сотрудник раздачи получает заказы окна в панели и собирает их по сводке порций. Шлюз «Все блюда есть?» имеет ветвь «нет»: сотрудник отмечает «Закончилось», и сервис скрывает позицию из меню (FR-02, FR-19). Сотрудник отмечает «Готов» для окна, и студент получает уведомление (FR-11). Студент приходит к отдельной точке выдачи, отмечает «Я на месте» и называет код из 4 цифр. Шлюз «Заказ найден?» имеет ветвь «нет»: сотрудник просит повторить код. Процесс заканчивается событием «Обед выдан» и отметкой «Выдан» (FR-18).

Вывод. В процессе «как будет» студент не стоит в общей очереди, а столовая знает заказы до перемены. Каждый оплаченный заказ сохраняется в сервисе.

3.1.3. Что изменилось в процессе

Таблица 3.1 сравнивает процессы «как есть» и «как будет» по формату «было — стало — что даёт». Каждая строка связана с одной задачей BG-1…BG-4. В столбце «Было» указан номер проблемы на рисунке 3.1.

Таблица 3.1 — Отличия процесса «как будет» от процесса «как есть»

№	Было («как есть»)	Стало («как будет»)	Что даёт	Задача
1	Студент стоит в общей очереди: в среднем 5,4 мин, максимум 8,5 мин (замеры, 5 перемен). Проблема 1.	Студент заранее выбирает окно получения и забирает заказ по коду у отдельной точки выдачи.	Время получения — до 2 минут.	BG-1
2	Блюдо заканчивается: 75 % студентов не нашли нужное блюдо (B6). Кухня считает остаток «на глаз» (П4). Проблема 3.	Меню с остатками публикуется до 08:00. Резерв держит остаток 10 минут. Сотрудник отмечает «Закончилось» одним действием.	Студент видит только блюда, которые есть. Меньше отмен «нет блюда».	BG-4
3	Оплата наличными или переводом на карту сотрудника, без чека (П4, П5). Проблема 4.	Оплата через Payme, Click или Uzum. Чек формирует платёжный сервис. Каждый заказ хранится в базе.	100 % оплаченных заказов в учёте. Отчёт продаж без ручного подсчёта.	BG-3
4	Кухня не знает спрос заранее и готовит «на глаз». Проблема 3.	Заказы сгруппированы по окнам. Сотрудник видит сводку порций. Лимит окна — 40 заказов.	Кухня собирает заказы до начала перемены.	BG-1
5	Студент узнаёт о наличии и готовности только у стойки (П2). Проблема 5.	Статус заказа виден в приложении. Уведомление «Готов» приходит в Telegram.	Студент приходит, когда заказ готов.	BG-1
6	64 % студентов уходили без обеда хотя бы 1 раз за 2 недели (B5). Проблема 2.	Предзаказ на любую перемену дня. Отмена с возвратом денег до статуса «Готовится».	Больше обедов через приложение, меньше риска потерять деньги.	BG-2

Главные отличия — первые три строки. Первое: очередь заменяет окно получения и код, поэтому время получения зависит от сотрудника, а не от длины очереди. Второе: остаток виден до заказа и держится резервом, поэтому студент не платит за блюдо, которого нет. Третье: оплата идёт через лицензированный платёжный сервис, поэтому у столовой появляется учёт и чек (C-02, C-03). Строки 4–6 поддерживают эти отличия: кухня знает спрос, студент знает статус, а отмена снижает риск предоплаты.

Вывод. Сервис меняет 6 шагов процесса. Каждое изменение работает на одну из задач BG-1… BG-4, а вместе они работают на цель BG-0.

3.1.4. Матрица RACI

Матрица RACI показывает, кто выполняет каждую работу процесса «как будет». R — выполняет, A — отвечает за результат, C — даёт совет, I — получает информацию. В каждой строке ровно одна буква A (таблица 3.2).

Таблица 3.2 — Матрица RACI процесса «как будет»

Работа процесса	Студент	Сотрудник раздачи	Администратор столовой	Платёжный сервис	Команда сервиса
Опубликовать меню дня с остатками до 08:00	I	C	A/R	—	—
Задать лимиты окон получения	I	C	A/R	—	—

Работа процесса	Студент	Сотрудник раздачи	Администратор столовой	Платёжный сервис	Команда сервиса
Оформить заказ и выбрать окно	A/R	—	I	—	—
Провести оплату и сформировать чек	R	—	I	A	C
Собрать заказы окна по сводке порций	I	R	A	—	—
Отметить, что блюдо закончилось	I	R	A	—	—
Выдать заказ по коду получения	C	R	A	—	—
Отменить заказ и вернуть деньги	R	I	A	R	C
Сверить отчёт продаж с выпиской	—	—	A/R	C	C
Поддерживать работу сервиса	—	I	I	—	A/R

Администратор столовой отвечает (A) за 7 работ из 10. Причина: деньги, меню и выдача — это работа столовой, а сервис только помогает. Платёжный сервис отвечает за проведение оплаты и чек, потому что по ограничению C-03 у столовой нет своей кассы. Студент отвечает только за свой заказ. Команда сервиса отвечает за работу сервиса: мониторинг, резервные копии и исправление ошибок. Сотрудник раздачи выполняет работу (R), но решения принимает администратор.

Вывод. Ответственность разделена без пересечений: в каждой строке одна роль A. Эту матрицу мы используем в договоре о пилоте со столовой.

3.2. UML-модели

Автор раздела: Шашкина Виктория (A) (3.2.1, 3.2.2); Фаткулбаянов Руслан (C) (3.2.3); Алиев Салим (B) (3.2.4)

В разделе 3.2 пять UML-диаграмм: диаграмма вариантов использования, две диаграммы деятельности, диаграмма последовательности для оплаты и диаграмма классов предметной области. Диаграммы описывают одну и ту же систему с разных сторон, поэтому мы используем одинаковые названия из глоссария главы 2.

3.2.1. Диаграмма вариантов использования

Диаграмма вариантов использования показывает, что делает с системой каждая роль (рисунок 3.4). На диаграмме три роли и три внешние системы. Таблица 3.3 связывает каждый вариант использования с историей US и требованиями FR.

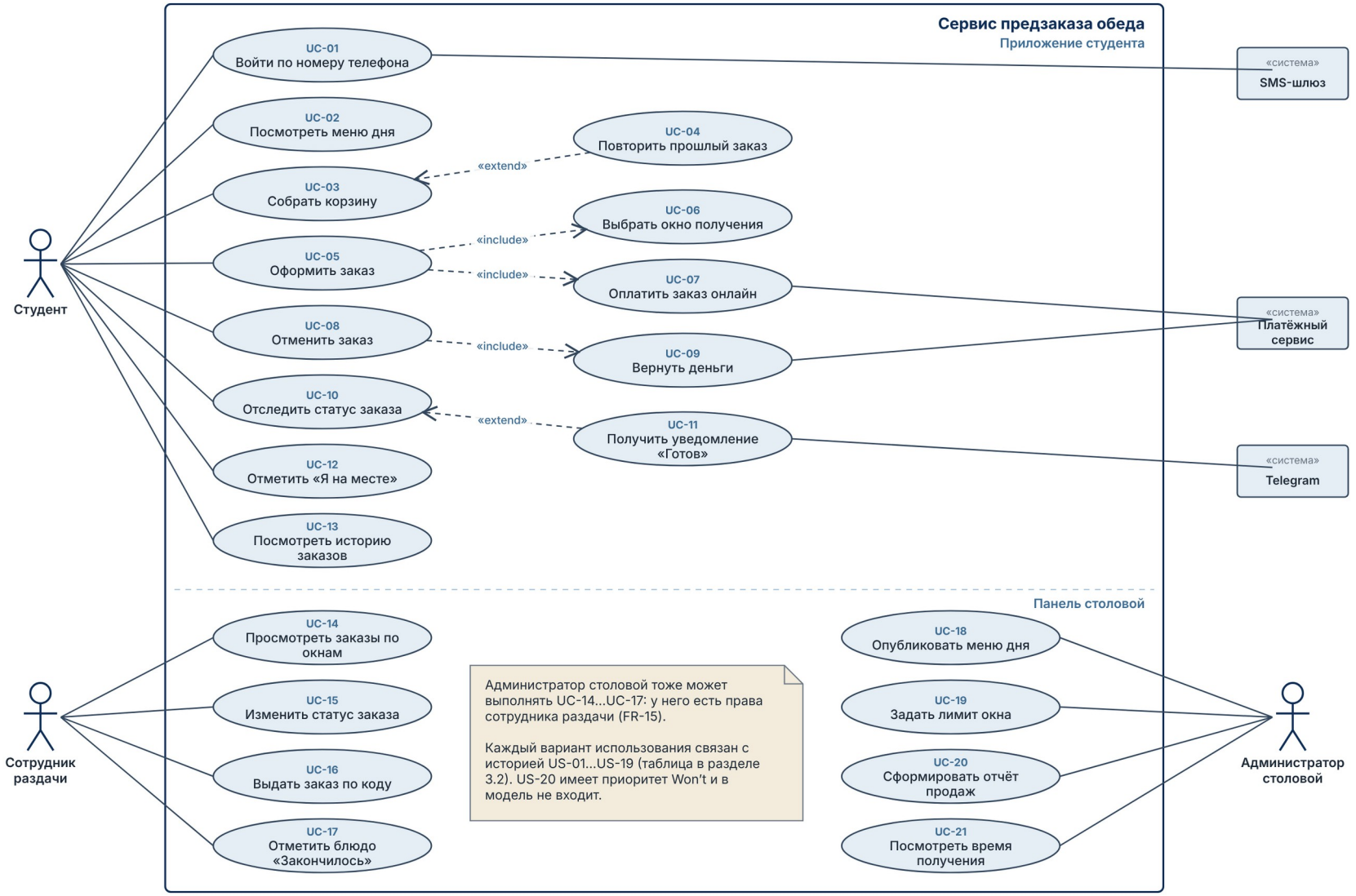


Рисунок 3.4 — Диаграмма вариантов использования (UML)

Таблица 3.3 — Варианты использования, истории и требования

UC	Название	Актор	История	FR	Связь
UC-01	Войти по номеру телефона	Студент	US-11	FR-15	SMS-шлюз
UC-02	Посмотреть меню дня	Студент	US-01	FR-01, FR-02	—
UC-03	Собрать корзину	Студент	US-02	FR-03	—
UC-04	Повторить прошлый заказ	Студент	US-09	FR-25	«extend» UC-03
UC-05	Оформить заказ	Студент	US-02, US-03, US-04	FR-08, FR-09, FR-23	«include» UC-06, UC-07
UC-06	Выбрать окно получения	Студент	US-03	FR-04, FR-05, FR-06	—
UC-07	Оплатить заказ онлайн	Студент	US-04	FR-07, FR-08	Платёжный сервис
UC-08	Отменить заказ	Студент	US-08	FR-13	«include» UC-09
UC-09	Вернуть деньги	Студент	US-08	FR-14	Платёжный сервис
UC-10	Отследить статус заказа	Студент	US-05	FR-10	—
UC-11	Получить уведомление «Готов»	Студент	US-06	FR-11	«extend» UC-10; Telegram
UC-12	Отметить «Я на месте»	Студент	US-07	FR-12, FR-27	—
UC-13	Посмотреть историю заказов	Студент	US-10	FR-24	—
UC-14	Просмотреть заказы по окнам	Сотрудник раздачи	US-12	FR-16	—
UC-15	Изменить статус заказа	Сотрудник раздачи	US-13	FR-17	—
UC-16	Выдать заказ по коду	Сотрудник раздачи	US-14	FR-18	—
UC-17	Отметить блюдо «Закончилось»	Сотрудник раздачи	US-15	FR-19	—
UC-18	Опубликовать меню дня	Администратор	US-16	FR-20	—
UC-19	Задать лимит окна	Администратор	US-17	FR-21	—
UC-20	Сформировать отчёт продаж	Администратор	US-18	FR-22	—
UC-21	Посмотреть время получения	Администратор	US-19	FR-26	—

На диаграмме 21 вариант использования: 13 у студента, 4 у сотрудника раздачи и 4 у администратора столовой. Отношение «include» показывает обязательную часть: заказ всегда включает выбор окна и оплату, отмена всегда включает возврат денег. Отношение «extend» показывает необязательную часть: повтор прошлого заказа расширяет сборку корзины, уведомление «Готов» расширяет отслеживание статуса. Внешние системы — SMS-шлюз, платёжный сервис и Telegram — связаны только с теми вариантами, где они нужны. Администратор столовой имеет права сотрудника раздачи, это показано заметкой на диаграмме. История US-20 имеет приоритет Won’t, поэтому у неё нет варианта использования.

Вывод. Все 19 историй первой версии покрыты вариантами использования. Каждый вариант использования ссылается на историю, поэтому трассировка не прерывается.

3.2.2. Диаграммы деятельности

Диаграммы деятельности показывают шаги двух главных сценариев с альтернативными ветвями. Рисунок 3.5 описывает оформление и оплату заказа. Рисунок 3.6 описывает получение заказа. Дорожки на диаграммах показывают, кто выполняет каждый шаг.

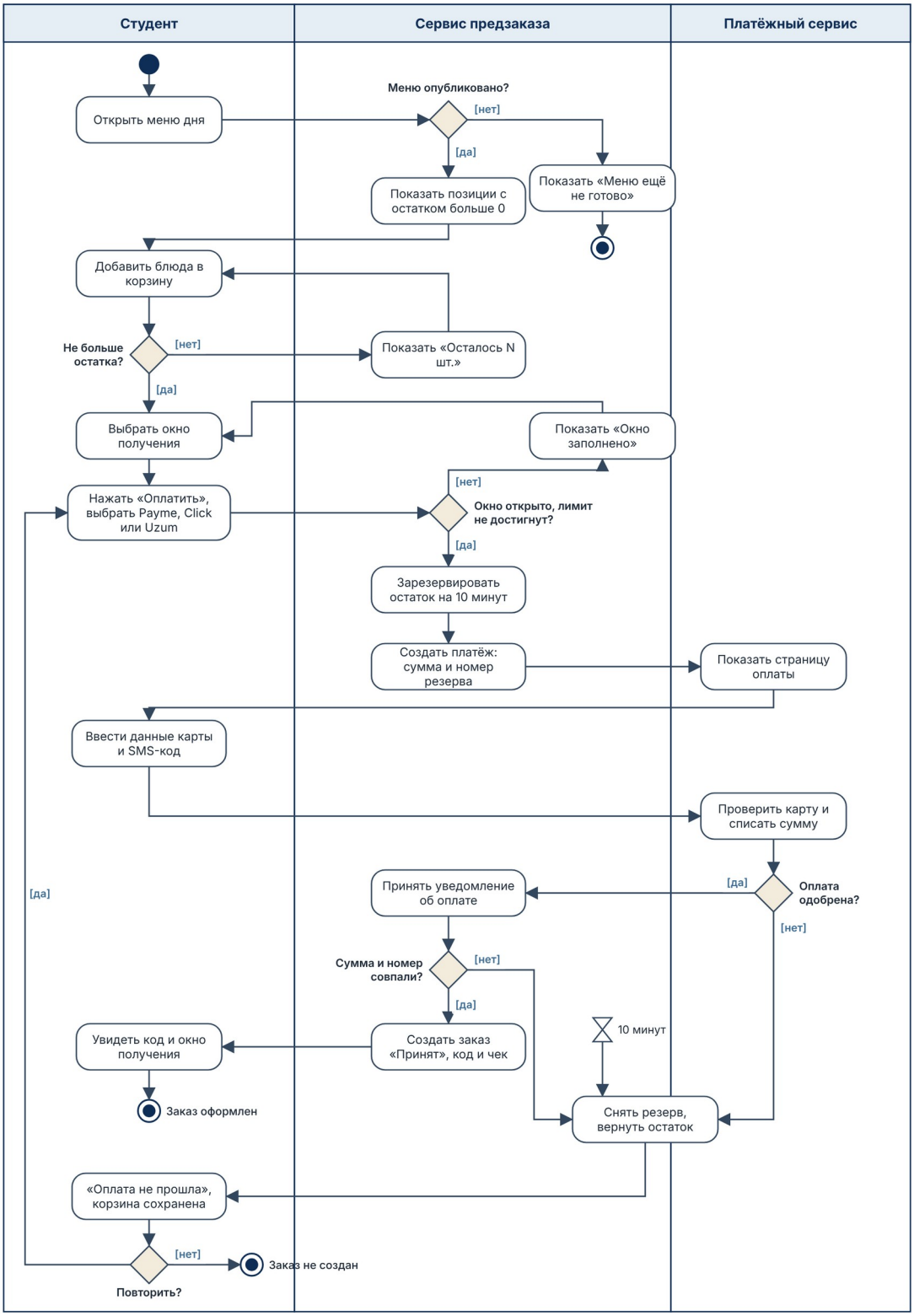


Рисунок 3.5 — Диаграмма деятельности «Оформить и оплатить заказ» (UML)

На рисунке 3.5 три дорожки: «Студент», «Сервис предзаказа» и «Платёжный сервис». Диаграмма имеет 5 альтернативных ветвей. Первая: меню ещё не опубликовано, студент видит сообщение «Меню ещё не готово». Вторая: остаток меньше нужного количества, сервис показывает

«Осталось N шт.». Третья: окно заполнено, студент выбирает другое окно. Четвёртая: платёжный сервис отклоняет оплату или сумма не совпала. Пятая: через 10 минут сервис снимает резерв и возвращает остаток. Во всех ветвях ошибки корзина сохраняется, и студент может повторить оплату.

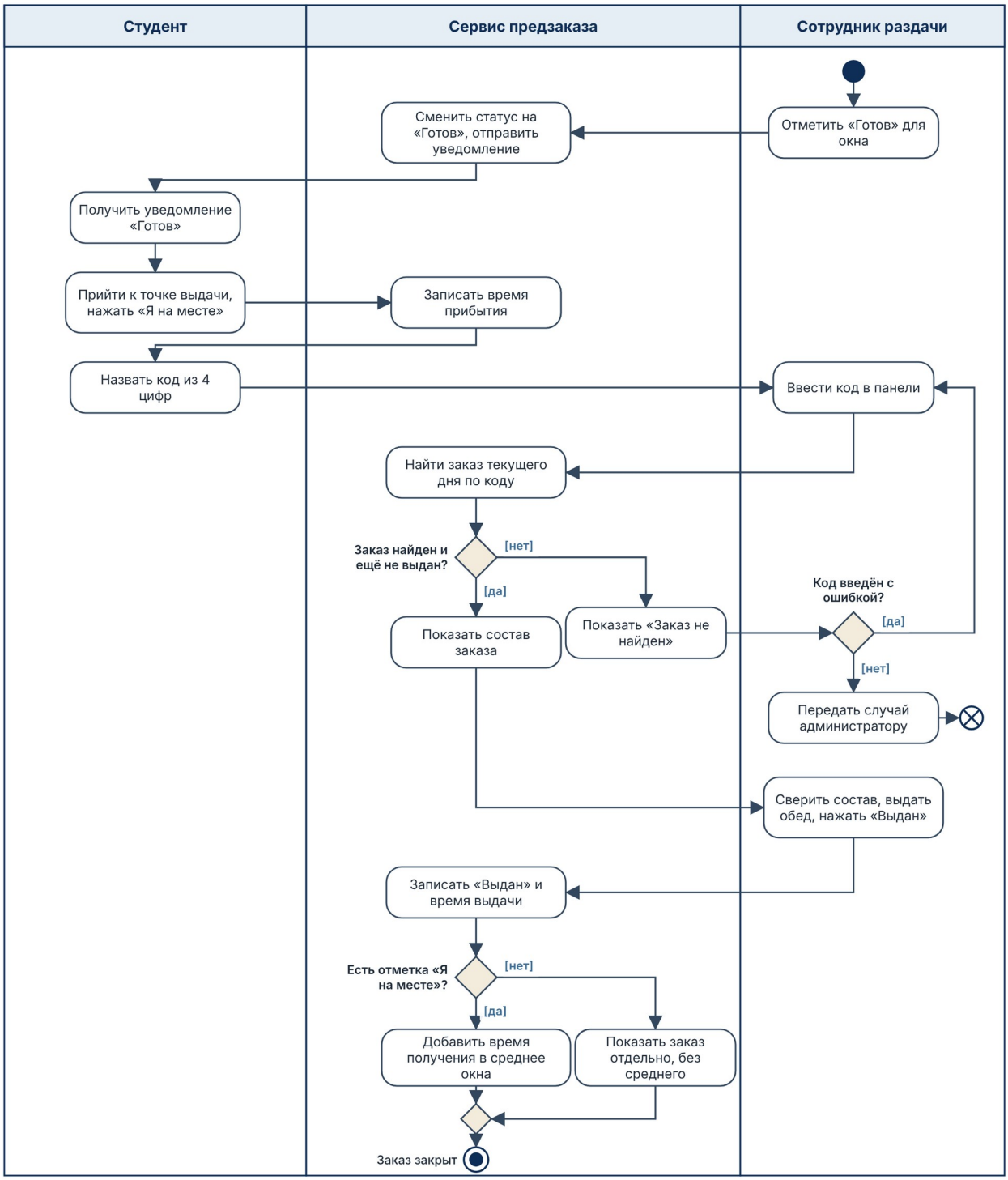


Рисунок 3.6 — Диаграмма деятельности «Получить заказ» (UML)

На рисунке 3.6 три дорожки: «Студент», «Сервис предзаказа» и «Сотрудник раздачи». Сервис проверяет, что заказ найден и ещё не выдан. Если заказ не найден, сотрудник проверяет, нет ли ошибки в коде, и просит повторить код. Если ошибки нет, сотрудник передаёт случай администратору (финальный узел потока). Если студент не отметил «Я на месте», сервис не учитывает заказ в среднем времени получения и показывает его отдельно (FR-26).

Вывод. Обе диаграммы показывают не только успешный путь, но и все ошибки, которые мы нашли в интервью и в процессе «как будет». Эти ветви мы используем для тест-кейсов на этапе 4.

3.2.3. Диаграмма последовательности: оплата заказа

Оплата — самая рискованная часть системы: в ней участвует внешний сервис и деньги студента. Диаграмма последовательности (рисунок 3.7) показывает сообщения между 6 участниками в порядке времени. Номера сообщений помогают ссылаться на шаги.

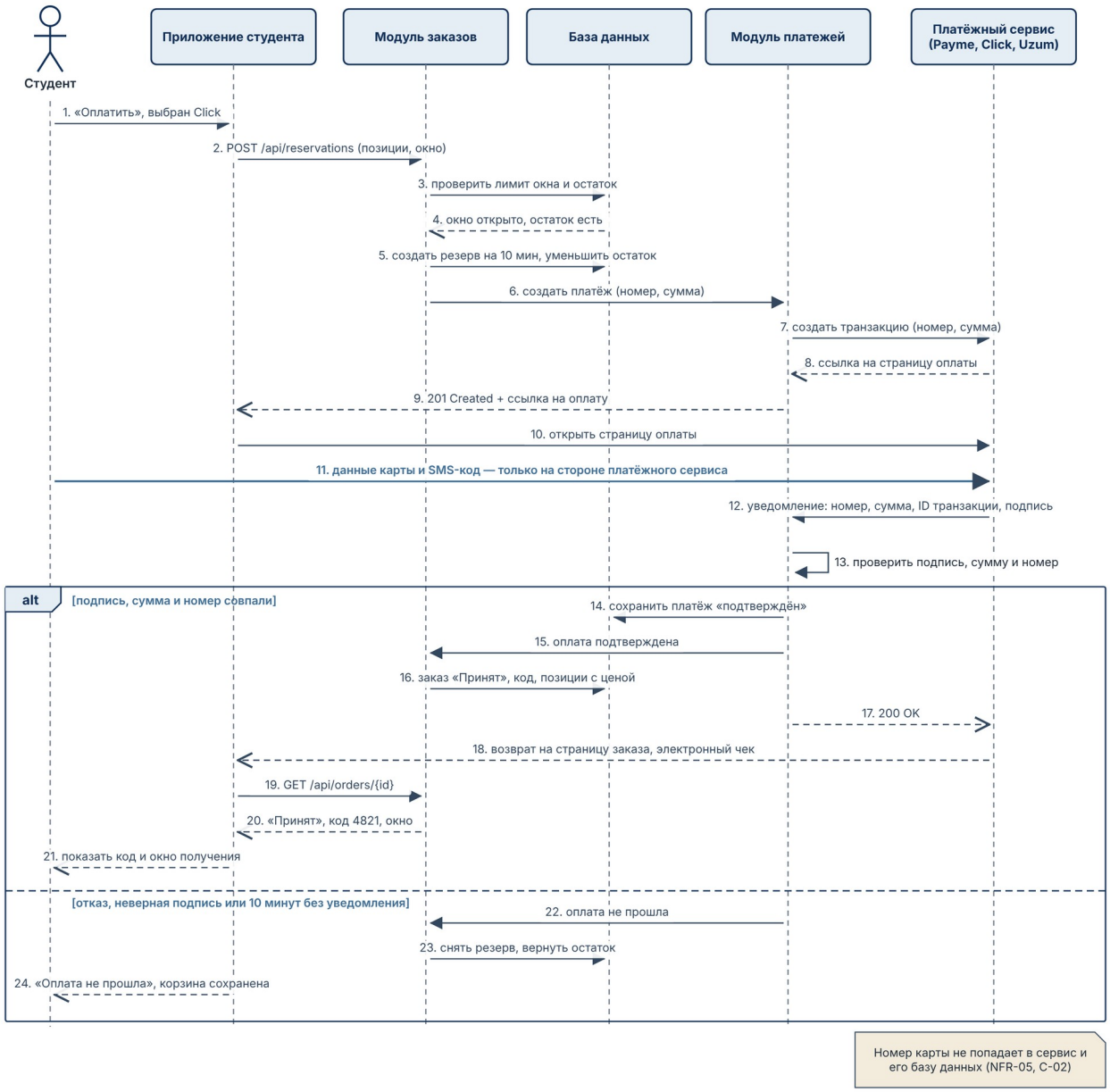


Рисунок 3.7 — Диаграмма последовательности «Оплатить заказ» (UML)

Приложение студента отправляет запрос POST /api/reservations. Модуль заказов в одной транзакции резервирует остаток на 10 минут и создаёт платёж. Студент переходит на страницу платёжного сервиса и вводит данные карты только там. Платёжный сервис отправляет уведомление с подписью. Модуль платежей проверяет подпись, сумму и номер резерва. Фрагмент alt имеет две ветви: если проверка успешна, модуль заказов создаёт заказ «Принят» и код получения; если пришёл отказ, подпись неверна или прошло 10 минут, сервис снимает резерв и показывает «Оплата не прошла».

Вывод. Номер карты не попадает в сервис и в его базу данных (NFR-05, C-02). Заказ создаётся только после проверенного уведомления (FR-08), поэтому сбой не создаёт неоплаченный заказ.

3.2.4. Диаграмма классов предметной области

Диаграмма классов показывает понятия предметной области, их атрибуты и связи (рисунок 3.8). Названия классов совпадают с терминами глоссария главы 2. Эту диаграмму мы используем как основу для схемы базы данных в разделе 3.3.

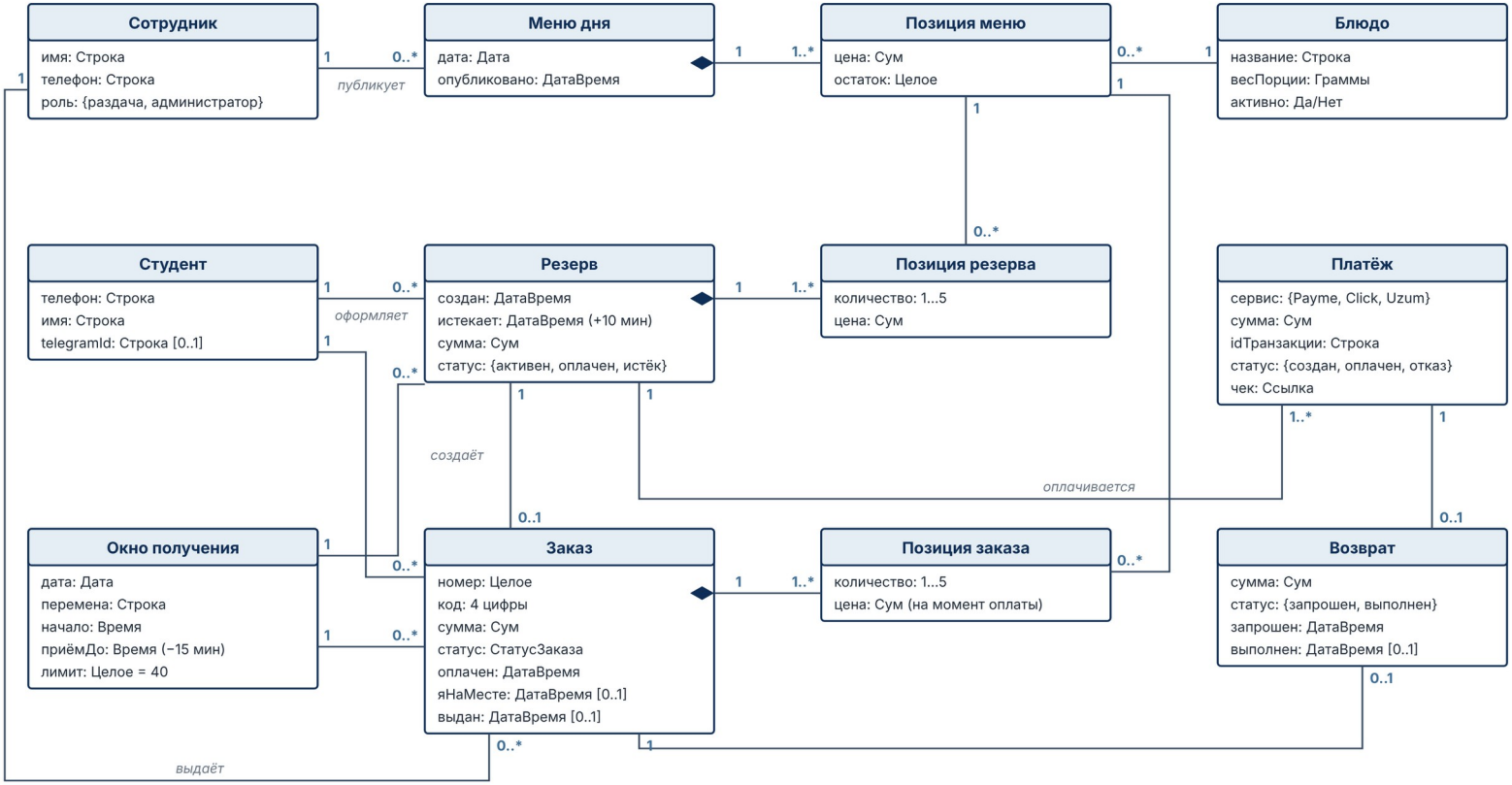


Рисунок 3.8 — Диаграмма классов предметной области (UML)

На диаграмме 12 классов. Композиция (закрашенный ромб) показывает, что позиция не живёт без своего целого: «Меню дня» — «Позиция меню», «Резерв» — «Позиция резерва», «Заказ» — «Позиция заказа». Кратность 1 — 0..1 между «Резерв» и «Заказ» показывает, что заказ появляется только после оплаты резерва, а истёкший резерв остаётся без заказа. «Окно получения» имеет атрибуты «лимит = 40» и «приёмДо» (начало окна минус 15 минут). «Позиция заказа» хранит цену на момент оплаты. Класс «Возврат» связан с заказом и показывает отмену с возвратом денег (US-08).

Вывод. Класс «Резерв» решает противоречие FR-08 и FR-09: остаток держится до оплаты, а заказ создаётся только после оплаты.

3.3. Модель данных

Автор раздела: Алиев Салим (В)

Модель данных переводит классы раздела 3.2.4 в таблицы PostgreSQL 16. В схеме 13 таблиц. Таблица 3.4 коротко описывает каждую таблицу и показывает, какие требования она выполняет. Рисунок 3.9 показывает физическую схему: первичные ключи (PK), внешние ключи (FK), типы полей и связи 1:N. Полное описание полей дано в приложении Г.

Таблица 3.4 — Таблицы базы данных

Таблица	Что хранит	Связи (FK)	Требования
staff	Сотрудники и администраторы, роль, хэш пароля	—	NFR-07
dishes	Справочник блюд: название, вес порции	—	FR-01, FR-20
daily_menus	Меню на дату, время публикации	staff	FR-20
menu_items	Позиции меню дня: цена, остаток	daily_menus, dishes	FR-01, FR-02, FR-19
students	Студенты: телефон, имя, Telegram	—	FR-15, FR-11
pickup_windows	Окна получения: время, приём до, лимит	—	FR-04, FR-05, FR-06, FR-21
reservations	Резерв на 10 минут, номер для оплаты	students, pickup_windows	FR-09
reservation_items	Позиции резерва: количество, цена	reservations, menu_items	FR-09
payments	Платежи: сервис, ID транзакции, сумма, чек	reservations	FR-07, FR-08, NFR-05
orders	Заказы: код, статус, время оплаты, «Я на месте», выдачи	reservations, students, pickup_windows, staff	FR-08, FR-10, FR-12, FR-18, FR-23, FR-27
order_items	Позиции заказа: количество, цена на момент оплаты	orders, menu_items	FR-23
refunds	Возвраты: сумма, статус, время	orders, payments	FR-13, FR-14
order_status_log	Журнал смен статуса заказа	orders, staff	FR-17, FR-26

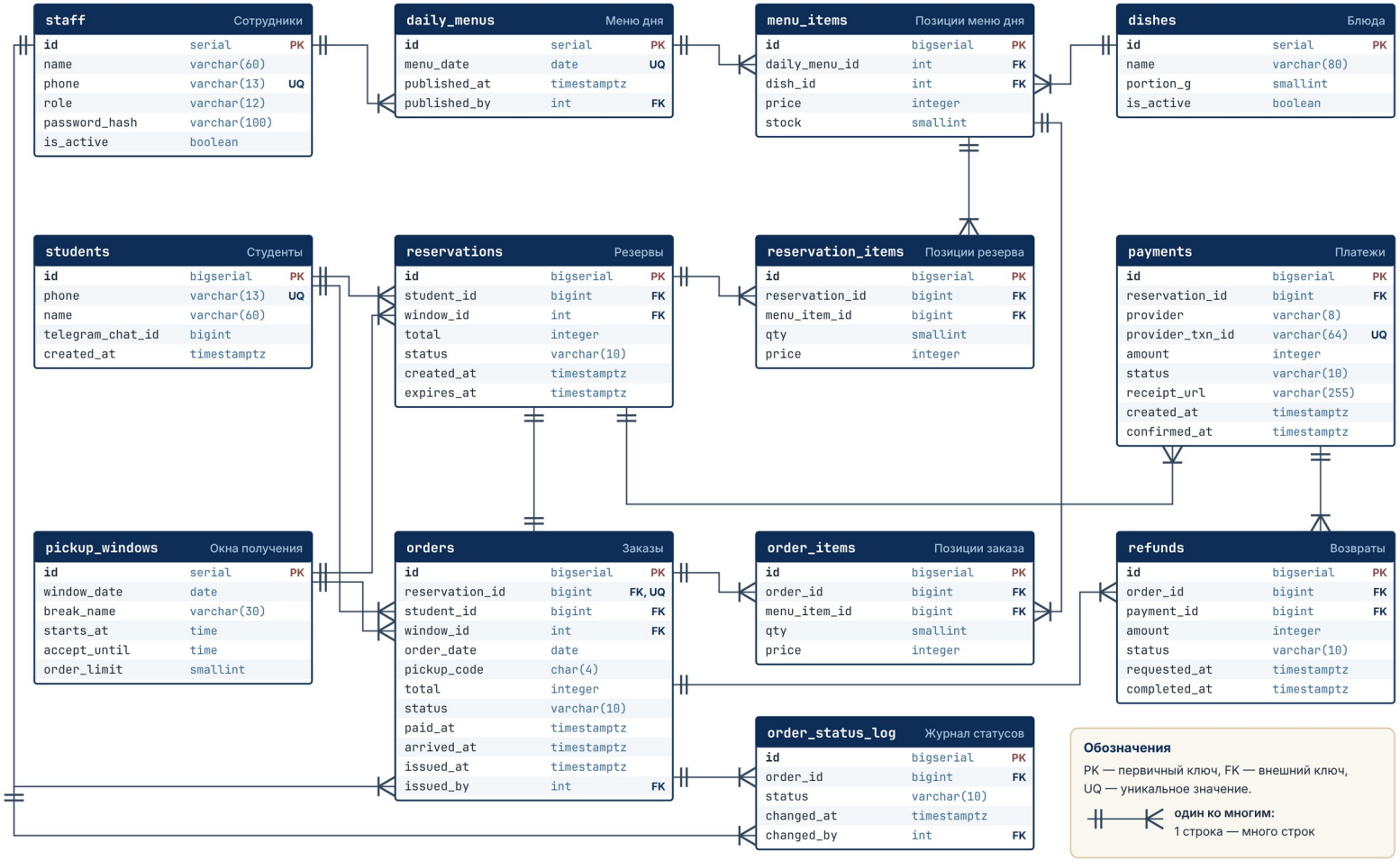


Рисунок 3.9 — Физическая схема базы данных (PostgreSQL 16)

В схеме 13 таблиц и 18 связей. Все связи, кроме одной, имеют тип 1:N: например, один заказ имеет много позиций заказа, одно окно — много заказов. Связь reservations — orders имеет тип 1:1, потому что из одного резерва появляется не больше одного заказа (UNIQUE на orders.reservation_id). Таблица позиций заказа order_items обязательна: она хранит состав заказа и позволяет построить отчёт продаж по позициям (FR-22). Ключи PK имеют тип serial или bigserial. Суммы хранятся в сумах как integer, потому что у сума нет дробной части в ценах столовой. Время хранится как timestamptz, чтобы не было ошибок с часовым поясом.

Схема находится в третьей нормальной форме, кроме 6 полей. Эти поля мы сознательно храним как копию (денормализация). Таблица 3.5 показывает причину для каждого поля.

Таблица 3.5 — Денормализация и её обоснование

Поле	Почему храним копию	Как держим значение верным
order_items.price	Администратор может изменить цену после оплаты (US-16). Заказ должен показать сумму, которую списали.	Сервис копирует menu_items.price в момент оплаты. После этого поле не меняется.
reservation_items.price	Сервис сравнивает сумму из уведомления платёжного сервиса с суммой резерва.	Копия цены в момент резерва. Резерв живёт не больше 10 минут.
orders.total	Ежедневная сверка с выпиской (BG-3, NFR-04) идёт без пересчёта по позициям.	Сумма записывается в одной транзакции с позициями заказа. Тест сверяет total и сумму qty × price.
menu_items.stock	Меню открывается быстрее, если не считать остаток по заказам (NFR-01).	Остаток меняется в одной транзакции с резервом (SELECT … FOR UPDATE). CHECK (stock ≥ 0).
orders.status	Панель и приложение читают текущий статус без чтения журнала.	Статус и строка order_status_log записываются в одной транзакции.
orders.order_date	Код получения уникален в пределах дня (FR-12).	Дата берётся из окна получения. UNIQUE (order_date, pickup_code).

Главный пример — цена в позиции заказа. Если хранить только ссылку на menu_items, то изменение цены изменит старые заказы и отчёт продаж. Поэтому order_items.price — это цена на момент оплаты. Каждую копию мы записываем в одной транзакции с исходными данными, поэтому копии не расходятся. Номер карты в схеме нет: таблица payments хранит только ID транзакции платёжного сервиса, сумму и ссылку на чек (NFR-05). Из персональных данных студента схема хранит только телефон, имя и chat_id Telegram; сервер находится в Узбекистане (C-01).

Вывод. Схема выполняет требования к данным из главы 2 и требования задания: больше 8 таблиц, таблица позиций заказа, PK, FK, типы и связи 1:N. Денормализация ограничена 6 полями, и для каждого поля есть правило, которое держит его верным. SQL-скрипт schema.sql мы запустили в PostgreSQL 16: он создаёт 13 таблиц и 18 внешних ключей без ошибок.

3.4. Архитектура

Автор раздела: Фаткулбаянов Руслан (С)

Архитектура показывает, из каких частей состоит система и как они связаны. Мы описываем 5 уровней: клиенты, шлюз, сервисы, данные и внешние системы. Сначала мы выбираем архитектурный стиль, потом описываем уровни, модули и интеграции.

3.4.1. Архитектурный стиль

Мы сравнили три стиля по критериям, которые важны для нашего продукта (таблица 3.6). Критерии взяты из ограничений C-01…C-05, нагрузки A-04 и требований NFR-02 и NFR-04.

Таблица 3.6 — Сравнение архитектурных стилей

Критерий	Монолит без модулей	Модульный монолит	Микросервисы
Скорость разработки командой из 3 человек	Высокая в начале, низкая потом	Высокая	Низкая
Стоимость размещения	1 сервер	1 сервер	5–8 сервисов, оркестратор
Надёжность платежа (одна транзакция)	Да	Да	Нужна распределённая транзакция
Сложность развёртывания	Низкая	Низкая	Высокая
Можно вынести модуль позже	Трудно	Да, по границам модулей	Уже вынесен
Подходит для 250 обедов в перемену (A-04)	Да	Да	Избыточно

Выбор: модульный монолит. У нас один небольшой продукт, команда из 3 человек и нагрузка около 250 обедов в большую перемену (A-04), поэтому микросервисы дают лишнюю сложность и стоимость. Резерв, платёж и заказ должны меняться в одной транзакции базы данных, а в монолите это просто (NFR-04). Модули имеют чёткие границы, поэтому позже мы можем вынести модуль платежей в отдельный сервис без переписывания.

Вывод. Модульный монолит на Python/Django — самый простой стиль, который выполняет требования к надёжности платежа и стоимости.

3.4.2. Уровни и модули

Рисунок 3.10 показывает уровни системы и внешние системы справа. Номера требований на рисунке показывают, какой модуль выполняет какое требование.

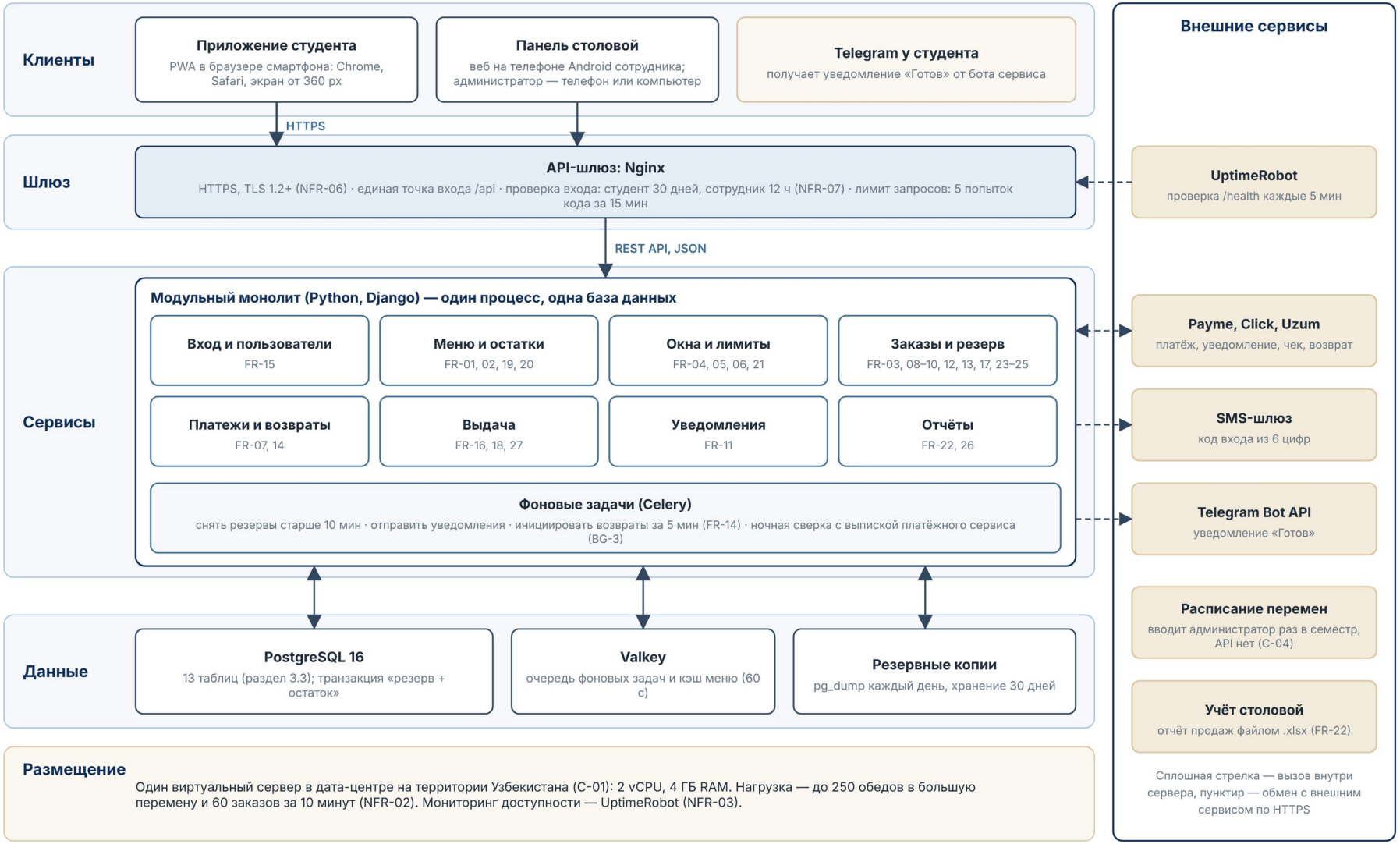


Рисунок 3.10 — Архитектура: уровни, модули и внешние системы

Клиенты: приложение студента (PWA в браузере смартфона), панель столовой на телефоне сотрудника и Telegram у студента. Шлюз Nginx принимает только HTTPS (TLS 1.2+), проверяет вход и ограничивает число попыток кода. Уровень сервисов — модульный монолит на Django с 8 модулями и фоновыми задачами Celery. Данные: PostgreSQL 16 для заказов, Valkey для кэша меню и очереди задач, ночные резервные копии. Сервер VPS (2 vCPU, 4 ГБ) находится в Узбекистане по ограничению C-01.

Таблица 3.7 показывает, что делает каждый модуль, какие требования он выполняет и с какими таблицами работает.

Таблица 3.7 — Модули сервиса

Модуль	Что делает	FR	Таблицы
Вход и пользователи	Вход по SMS-коду, сессии студента и сотрудника	FR-15	students, staff
Меню и остатки	Меню дня, остатки, отметка «Закончилось»	FR-01, FR-02, FR-19, FR-20	daily_menus, menu_items, dishes
Окна и лимиты	Окна по расписанию, закрытие за 15 минут, лимит	FR-04, FR-05, FR-06, FR-21	pickup_windows
Заказы и резерв	Корзина, резерв 10 минут, заказ после оплаты, статусы, код, история	FR-03, FR-08…FR-10, FR-12, FR-13, FR-17, FR-23…FR-25	reservations, reservation_items, orders, order_items, order_status_log
Платежи и возвраты	Payme, Click, Uzum; проверка подписи; возврат	FR-07, FR-14	payments, refunds
Выдача	Сводка порций, поиск по коду, «Я на месте», «Выдан»	FR-16, FR-18, FR-27	orders
Уведомления	Сообщение «Готов» в Telegram за 30 секунд	FR-11	students
Отчёты	Отчёт продаж .xlsx, время получения по окнам	FR-22, FR-26	orders, order_items, payments
Фоновые задачи (Celery)	Снять истёкший резерв, отправить уведомление, выполнить возврат	FR-09, FR-11, FR-14	reservations, refunds

Каждый модуль владеет своими таблицами. Модуль может читать таблицы другого модуля только через его функции, а не напрямую. Это правило держит границы модулей. Таблица 3.8 показывает, как архитектура выполняет нефункциональные требования.

Таблица 3.8 — Нефункциональные требования и решения в архитектуре

NFR	Требование	Решение в архитектуре
NFR-01	Меню ≤ 2 с	Меню дня хранится в кэше Valkey. Nginx отдаёт файлы приложения. Индекс по menu_date.
NFR-02	«Принят» ≤ 5 с после оплаты	Модуль платежей обрабатывает уведомление сразу. Заказ создаётся в одной транзакции.
NFR-03	Доступность ≥ 99 %	UptimeRobot проверяет сервис каждые 5 минут. Сервер в Узбекистане, автоматический перезапуск.
NFR-04	0 потерянных оплаченных заказов	UNIQUE на provider_txn_id: повторное уведомление не создаёт второй заказ. Сверка с выпиской каждый день. Резервная копия pg_dump каждую ночь, хранение 30 дней.
NFR-05	Нет данных карты на сервере	Студент вводит карту на странице платёжного сервиса. В базе только ID транзакции и сумма.
NFR-06	HTTPS, TLS 1.2+	Nginx с сертификатом Let’s Encrypt, TLS 1.2 и 1.3, HSTS.
NFR-07	Панель только после входа	Роли «раздача» и «администратор». Сессия сотрудника ≤ 12 часов.
NFR-08	Первый заказ ≤ 90 с	Путь из 4 экранов с одной главной кнопкой. Проверено тестом прототипа (раздел 3.5.2).
NFR-09	Выдача ≤ 10 с	Крупная цифровая клавиатура и одна кнопка «Выдать заказ».

NFR	Требование	Решение в архитектуре
NFR-10	Смартфоны от 360 px, Android 10+, iOS 15+	Приложение PWA в браузере. Панель столовой — то же PWA на телефоне сотрудника.

Вывод. Каждое требование NFR-01…NFR-10 имеет решение в архитектуре. Требования NFR-08 и NFR-09 мы проверили на прототипе, остальные проверим тестами на этапе 4.

3.4.3. Интеграции и риски

Сервис работает с 6 внешними системами. Таблица 3.9 показывает для каждой системы назначение, данные, риски и меры против рисков.

Таблица 3.9 — Интеграции с внешними системами

Сервис	Зачем	Какие данные	Риски	Что делаем
Payme, Click, Uzum (Merchant API)	Оплата, возврат, фискальный чек (FR-07, FR-14, C-02, C-03)	Передаём: номер резерва, сумму. Получаем: ID транзакции, статус, ссылку на чек.	Сервис недоступен. Поддельное или повторное уведомление.	3 сервиса на выбор. Проверка подписи и суммы. UNIQUE на ID транзакции. Таймер 10 минут снимает резерв.
SMS-шлюз (например, Eskiz)	Код входа (FR-15)	Номер телефона, текст с кодом	SMS задерживается. Перебор кодов. Стоимость SMS.	Повтор через 60 с. Не больше 5 попыток за 15 минут. Код действует 5 минут.
Telegram Bot API	Уведомление «Готов» (FR-11)	chat_id, номер заказа, окно	Студент не подключил бот. Telegram недоступен.	Статус всегда виден в приложении (US-05).
Расписание перемен	Окна получения (C-04)	Время начала перемен	Расписание изменилось.	Администратор меняет окна в панели.
Учёт столовой (.xlsx)	Отчёт продаж (FR-22)	Число заказов, суммы, порции	Ошибка в выгрузке.	Сверка с выпиской платёжного сервиса (BG-3).
UptimeRobot	Мониторинг доступности (NFR-03)	Адрес проверки, без персональных данных	Ложная тревога.	Тревога только после 2 неудачных проверок подряд.

Самый большой риск — платёжный сервис. Если уведомление не пришло, деньги студента могут быть списаны, а заказа нет. Поэтому сервис проверяет подпись и сумму каждого уведомления, а повторное уведомление не создаёт второй заказ. Ежедневная сверка с выпиской находит расхождения. Второй риск — один сервер: если сервер не работает, заказы не принимаются. Мы ставим мониторинг, ночные резервные копии и проверяем восстановление за 1 час. Третий риск — слабый мобильный интернет на цокольном этаже (28 % ответили «с перебоями», B9). Поэтому приложение сохраняет код получения в телефоне, и код виден без интернета.

Таблица 3.10 показывает главные методы REST API. Эти методы используют диаграмма последовательности (рисунок 3.7) и прототип.

Таблица 3.10 — Главные методы REST API

Метод и адрес	Что делает	FR
POST /api/auth/sms, POST /api/auth/verify	Отправить SMS-код, проверить код	FR-15
GET /api/menu/today	Меню дня с остатками	FR-01, FR-02
GET /api/windows/today	Окна получения и свободные места	FR-04…FR-06
POST /api/reservations	Создать резерв на 10 минут и ссылку на оплату	FR-09, FR-07

Метод и адрес	Что делает	FR
POST /api/payments/{provider}/callback	Принять уведомление платёжного сервиса	FR-08, FR-23
GET /api/orders/{id}	Статус, код и чек заказа	FR-10, FR-12
POST /api/orders/{id}/arrived	Отметить «Я на месте»	FR-27
POST /api/orders/{id}/cancel	Отменить заказ и начать возврат	FR-13, FR-14
GET /api/staff/windows/{id}/orders	Заказы окна и сводка порций	FR-16
PATCH /api/staff/orders/{id}/status	Изменить статус заказа	FR-17
POST /api/staff/orders/issue	Найти заказ по коду и отметить «Выдан»	FR-18
POST /api/staff/menu-items/{id}/sold-out	Поставить остаток 0	FR-19
PUT /api/admin/menu/{date}, PUT /api/admin/windows/{id}	Опубликовать меню, задать лимит окна	FR-20, FR-21
GET /api/admin/reports/sales	Отчёт продаж за период (.xlsx)	FR-22, FR-26

Вывод. Для каждой интеграции есть мера против риска. Платёж защищён проверкой подписи, уникальным ID транзакции, таймером резерва и ежедневной сверкой.

3.5. Прототип интерфейса

Автор раздела: Алиев Салим (В)

Прототип показывает главный путь студента и экран сотрудника. В прототипе 7 связанных экранов, среди них экран ошибки оплаты. Прототип сделан в Figma; ссылка дана на титульном листе и в README репозитория. Ссылка открывается без входа в Figma (режим «Anyone with the link can view»).

3.5.1. Экраны и переходы

Рисунок 3.11 показывает все экраны и переходы между ними. Сплошная стрелка — переход по кнопке, пунктирная — переход между устройствами студента и сотрудника. Экран ошибки выделен рамкой.

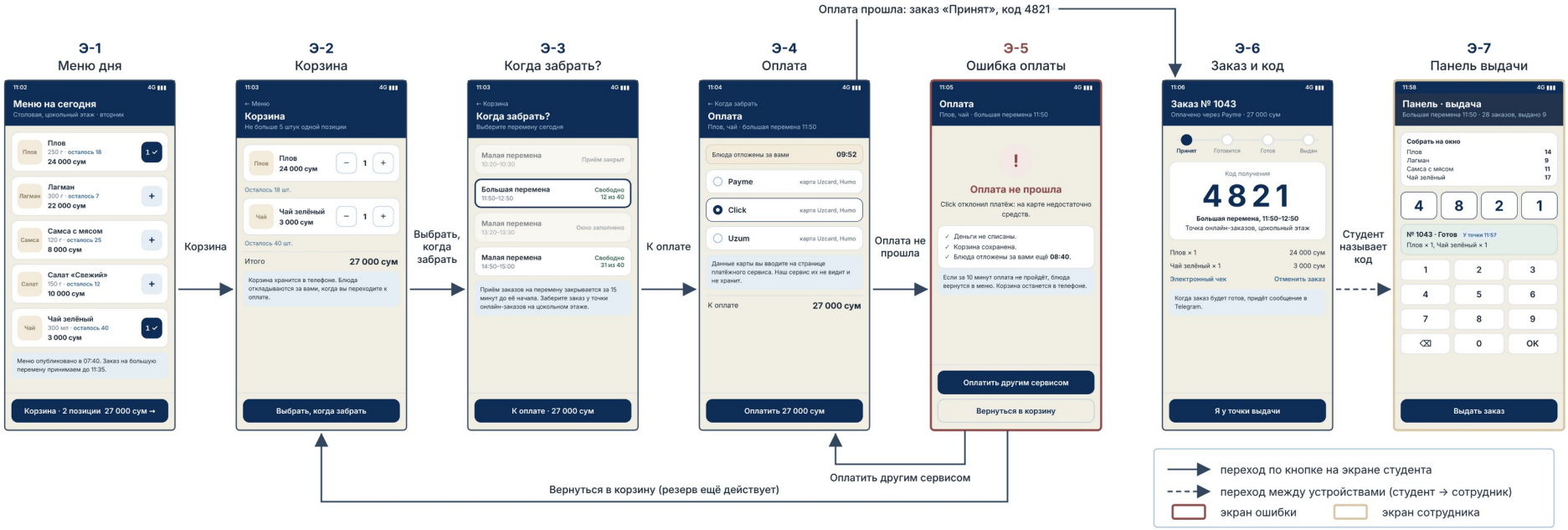


Рисунок 3.11 — Экраны прототипа и переходы (версия 2)

Студент проходит путь Э-1 → Э-2 → Э-3 → Э-4. Если оплата прошла, студент видит экран заказа Э-6 с кодом 4821. Если оплата не прошла, студент видит экран ошибки Э-5. С экрана ошибки есть два пути: оплатить другим сервисом (возврат на Э-4) или вернуться в корзину (Э-2), пока резерв действует. На экране Э-7 сотрудник вводит код, который называет студент, и выдаёт заказ.

Таблица 3.11 связывает каждый экран с вариантами использования, историями и требованиями.

Таблица 3.11 — Экраны прототипа и трассировка

Экран	Что делает пользователь	UC	US	FR
Э-1 Меню дня	Смотрит блюда, цены и остатки, добавляет блюдо в корзину	UC-02, UC-03	US-01, US-02	FR-01, FR-02, FR-03
Э-2 Корзина	Меняет количество (1–5), видит итог	UC-03	US-02	FR-03
Э-3 Когда забрать?	Выбирает окно; видит закрытые и заполненные окна	UC-06	US-03	FR-04, FR-05, FR-06
Э-4 Оплата	Выбирает Payme, Click или Uzum; видит таймер резерва	UC-07	US-04	FR-07, FR-08, FR-09
Э-5 Ошибка оплаты	Видит причину отказа; платит другим сервисом или возвращается в корзину	UC-07	US-04	FR-08, FR-09
Э-6 Заказ и код	Видит статус, код, окно и чек; отменяет заказ; отмечает «Я у точки выдачи»	UC-08, UC-10, UC-12	US-05, US-07, US-08	FR-10, FR-12, FR-13, FR-27
Э-7 Панель выдачи	Сотрудник видит сводку порций, вводит код и выдаёт заказ	UC-14, UC-16	US-12, US-14	FR-16, FR-18

Экраны покрывают 9 историй из 19: все истории главного пути студента и главные истории сотрудника. Экраны входа (US-11), меню администратора (US-16) и отчёта (US-18) не входят в 7 экранов. Мы нарисуем их на этапе 4, когда начнём разработку.

Вывод. Прототип показывает главный путь и ошибку оплаты, а каждый экран ссылается на UC и US.

3.5.2. Проверка прототипа на 5 участниках

Мы проверили первую версию прототипа на 5 участниках. Участники: У1 — студент 1 курса (Android), У2 — студентка 2 курса (iPhone), У3 — студент 3 курса (Android), У4 — студентка 4 курса (Android), У5 — студент 2 курса (iPhone, раньше не платил через Click). Метод: участник получает задачу, говорит вслух, ведущий молчит и записывает время секундомером. Сценарий и бланк даны в приложении Д. Данные проверки — модельные: команда составила их по сценарию, и на этапе 4 мы повторим проверку с реальными участниками. Таблица 3.12 показывает задачи.

Таблица 3.12 — Задачи проверки

Задача	Текст для участника	Успех	Мера
Т1	Закажите плов и зелёный чай на большую перемену. Оплатите через Click.	Участник нажал «Оплатить» на экране Э-4	NFR-08: ≤ 90 с у 4 из 5
Т2	Оплата не прошла. Оплатите заказ другим сервисом.	Участник открыл экран заказа Э-6	Успех 5 из 5
Т3	Найдите код получения и отметьте, что вы у точки выдачи.	Участник нажал «Я у точки выдачи»	Успех 5 из 5
Т4	Вы — сотрудник раздачи. Выдайте заказ по коду, который назовёт ведущий (2 раза).	Участник нажал «Выдать заказ» для верного заказа	NFR-09: ≤ 10 с

Таблица 3.13 показывает время каждого участника, успех и трудности.

Таблица 3.13 — Результаты проверки (время в секундах)

Задача	У1	У2	У3	У4	У5	Успех	Трудность
Т1, с	58	71	64	49	104	5 из 5; ≤ 90 с — 4 из 5	У3 и У5 не поняли слово «Окно». У2 не заметила, сколько времени держится резерв.

Задача	У1	У2	У3	У4	У5	Успех	Трудность
Т2, с	18	25	21	15	32	5 из 5	У2, У4 и У5 не были уверены, списались ли деньги.
Т3, с	6	9	7	5	12	5 из 5	У1 и У5 искали кнопку «Я на месте». У4: код мелкий, его трудно показать.
Т4, с (среднее из 2)	7	8	6	6	9	10 из 10 выдач	У3 ввёл неверную цифру, а стереть её было нельзя.

Все участники выполнили все задачи. Задачу Т1 4 участника из 5 выполнили быстрее 90 секунд, поэтому NFR-08 выполнено. У5 потратил 104 секунды: он искал, где выбрать перемену, потому что не понял слово «Окно». Задачу Т4 участники выполнили за 6–9 секунд, поэтому NFR-09 (≤ 10 с) выполнено. Самая частая трудность — экран ошибки: 3 участника из 5 не были уверены, списались ли деньги. Таблица 3.14 показывает исправления.

Таблица 3.14 — Исправления после проверки

№	Проблема (сколько участников)	Исправление в версии 2	Экран	Эвристика
1	Слово «Окно» непонятно (2 из 5)	Заголовок «Когда забрать?», названия перемен и время	Э-3	2
2	Не видно, сколько времени держится резерв (1 из 5)	Плашка «Блюда отложены за вами» с таймером	Э-4	1
3	Неясно, списаны ли деньги после отказа (3 из 5)	Блок «Деньги не списаны. Корзина сохранена. Блюда отложены ещё мм:сс»	Э-5	9
4	Кнопку «Я на месте» искали (2 из 5)	Текст «Я у точки выдачи», главная кнопка внизу экрана	Э-6	2, 4
5	Код мелкий (1 из 5)	Код крупным шрифтом 64 px	Э-6	8
6	Нельзя стереть цифру кода (1 из 5)	Клавиши «⌫» и «OK» на клавиатуре панели	Э-7	3

Вывод. Проверка нашла 6 проблем, и все они исправлены в версии 2 прототипа. Рисунок 3.11 показывает версию 2.

3.5.3. Самопроверка по 10 эвристикам Нильсена

После исправлений мы проверили версию 2 по 10 эвристикам Якоба Нильсена (таблица 3.15). Для каждой эвристики мы записали, как она выполнена и на каком экране.

Таблица 3.15 — Самопроверка по эвристикам Нильсена

№	Эвристика	Как выполнено	Экраны	Оценка
1	Видимость состояния системы	Шаги статуса «Принят → Готовится → Готов → Выдан», таймер резерва, «осталось N»	Э-1, Э-4, Э-6	Да
2	Соответствие реальному миру	Слова студента: «большая перемена», «точка выдачи», суммы в сумах	Э-3, Э-6	Да
3	Свобода и контроль пользователя	Кнопка «←» на каждом шаге, отмена заказа до «Готовится», стирание цифры	Э-2…Э-4, Э-6, Э-7	Да
4	Единообразие и стандарты	Одна главная кнопка внизу каждого экрана, одинаковые цвета и шрифты	Все	Да
5	Предотвращение ошибок	Закрытые и заполненные окна нельзя выбрать; не больше 5 штук; резерв остатка	Э-2, Э-3	Да
6	Узнавание вместо запоминания	Состав и окно видны на экранах оплаты и заказа; код всегда на экране заказа	Э-4, Э-6	Да

№	Эвристика	Как выполнено	Экраны	Оценка
7	Гибкость и эффективность	Повтор прошлого заказа (US-09, Could) пока не нарисован	—	Частично: добавим на Э-1
8	Эстетичный и минималистичный дизайн	Одна задача на экране, спокойные цвета, нет лишнего текста	Все	Да
9	Помощь при ошибках	Экран ошибки: причина, что с деньгами, два пути дальше	Э-5	Да
10	Справка и документация	Короткие подсказки о 15 минутах и о данных карты; раздела «Помощь» нет	Э-3, Э-4	Частично: добавим «Помощь» в профиль

Прототип выполняет 8 эвристик из 10. Эвристика 7 выполнена частично: повтор прошлого заказа (US-09) имеет приоритет Could, и мы добавим его на экран Э-1. Эвристика 10 выполнена частично: подсказки есть на экранах, но нет раздела «Помощь». Обе доработки не мешают главному пути.

Вывод. Прототип готов к защите: 7 экранов, экран ошибки, проверка на 5 участниках и самопроверка по Нильсену.

3.6. Выбор платформы и стоимость владения

Автор раздела: Фаткулбаянов Руслан (С)

Перед разработкой мы сравниваем три способа сделать сервис. Для каждого способа мы считаем стоимость владения за 3 года (TCO) и проверяем ограничения главы 2. Таблица 3.16 сравнивает способы по главным критериям.

Таблица 3.16 — Варианты платформы

Критерий	А. No-code (Bubble)	Б. Готовая SaaS-платформа заказов	В. Своя разработка на открытом стеке
Данные в Узбекистане (С-01)	Нет: серверы платформы за рубежом	Не гарантировано	Да: VPS в Узбекистане
Payme, Click, Uzum (С-02)	Только через доработку API	Зависит от платформы	Да, по Merchant API
Резерв 10 минут (FR-09), код (FR-12), «Я на месте» (FR-27)	Частично	Нет	Да
Срок до пилота	1–1,5 месяца	2–3 недели	3–4 месяца
Зависимость от поставщика	Высокая	Высокая	Низкая: открытые лицензии
TCO за 3 года (таблица 3.17)	≈ 83,7 млн сум	≈ 33,9 млн сум	≈ 102,3 млн сум

Вариант А (no-code) быстрый, но данные хранятся на серверах платформы за рубежом, а это нарушает С-01. Вариант Б (готовая SaaS-платформа) самый дешёвый, но в нём нет резерва остатка, кода получения и отметки «Я на месте», а это требования Must. Вариант В дороже, но только он выполняет все ограничения и требования Must. Таблица 3.17 показывает расчёт TCO.

Таблица 3.17 — Стоимость владения за 3 года (TCO)

Статья затрат за 3 года, сум	А. No-code	Б. SaaS	В. Своя разработка
Разработка и настройка (100 000 сум/ч)	12 000 000 (120 ч)	2 000 000 (20 ч)	48 000 000 (400 ч + 80 ч тестов)
Платформа или сервер	31 050 000 ($69/мес)	21 600 000 (600 000/мес)	9 000 000 (VPS 250 000/мес)
Домен	600 000	входит	600 000
SMS-коды (≈ 500 SMS/мес)	1 800 000	входит	1 800 000
Сертификат и мониторинг	0	0	0 (Let’s Encrypt, UptimeRobot)
Поддержка (6 ч/мес; SaaS — 2 ч/мес)	21 600 000	7 200 000	21 600 000
Развитие (40 ч/год; no-code — 30 ч/год)	9 000 000	0 (нельзя менять)	12 000 000
Резерв на скрытые затраты, 10 %	7 605 000	3 080 000	9 300 000
Итого TCO	**83 655 000**	**33 880 000**	**102 300 000**

Допущения расчёта. Курс: 1 USD = 12 500 сум. Час работы разработчика — 100 000 сум. VPS в Узбекистане (2 vCPU, 4 ГБ) — около 250 000 сум в месяц, домен — около 200 000 сум в год (тарифы хостингов Узбекистана, ноябрь 2026). SMS-код — около 100 сум за сообщение. Bubble Starter — $69 в месяц (сайт тарифов Bubble, 2026). Цена SaaS-платформы — оценка команды. Скрытые затраты (миграция, обучение сотрудников, простои) мы учли резервом 10 %.

Окупаемость. При 50 онлайн-заказах в день (BG-2: 20 % от 250 обедов), среднем чеке 25 000 сум и комиссии 5 % (A-07) столовая платит около 1 375 000 сум в месяц. За 3 учебных года (26 платных

месяцев после бесплатного пилотного месяца) это около 36 млн сум. Одна столовая не окупает вариант В. Вариант В окупается при 3 столовых и больше (3 × 35,75 ≈ 107 млн сум > 102,3 млн сум), если затраты на разработку почти не растут. Этот вывод мы добавляем в бизнес-модель (раздел 1.3).

Вариант В использует компоненты с открытыми лицензиями. Таблица 3.18 показывает лицензии и их требования.

Таблица 3.18 — Лицензии компонентов

Компонент	Роль	Лицензия	Что требует лицензия
React, Vite	Приложение студента (PWA), сборка	MIT	Сохранить текст лицензии
Django, Django REST framework	Сервер, REST API	BSD-3-Clause	Сохранить текст; не использовать имя авторов для рекламы
Celery	Фоновые задачи	BSD-3-Clause	Как выше
psycopg 3	Драйвер PostgreSQL	LGPL-3.0	Используем без изменений как библиотеку; свой код не открываем
Gunicorn	Сервер приложения	MIT	Сохранить текст лицензии
PostgreSQL 16	База данных	PostgreSQL License	Сохранить текст лицензии
Valkey	Кэш и очередь задач	BSD-3-Clause	Как выше. Redis не берём: с 2024 года его лицензии RSALv2, SSPL и AGPLv3 строже BSD
Nginx	Шлюз, HTTPS	BSD-2-Clause	Сохранить текст лицензии
aiogram	Бот Telegram	MIT	Сохранить текст лицензии
Let’s Encrypt	Сертификат TLS	Бесплатный сервис	Обновлять сертификат автоматически (certbot)
UptimeRobot, Figma	Мониторинг, прототип	Закрытые, бесплатный план	Ограничения плана; не открытый код

Все серверные компоненты имеют разрешительные лицензии (MIT, BSD, PostgreSQL License). Они разрешают коммерческое использование и не требуют открывать наш код. Драйвер psycopg имеет лицензию LGPL-3.0: мы используем его без изменений как библиотеку, поэтому наш код остаётся закрытым. Мы выбрали Valkey вместо Redis, потому что в 2024 году Redis сменил лицензию BSD на более строгие (RSALv2 и SSPL, позже также AGPLv3).

Вывод. Мы выбираем вариант В: свою разработку на открытом стеке. Он дороже по TCO, но только он выполняет С-01, С-02 и требования Must. Для окупаемости сервису нужны 3 столовые и больше.

3.7. Трассировка: дополнение к матрице 2.7

Автор раздела: Шашкина Виктория (А)

Матрица 2.7 связывает задачи BG, истории US, требования FR и тест-кейсы TC. В главе 3 мы добавляем в неё три столбца: вариант использования, экран прототипа и таблицы базы данных (таблица 3.19). Номера TC совпадают с главой 2.

Таблица 3.19 — Трассировка BG → US → UC → экран → БД → TC

US	Приоритет	BG	UC	Экран	Таблицы БД	TC
US-01	Must	BG-2, BG-4	UC-02	Э-1	daily_menus, menu_items, dishes	TC-001…004
US-02	Must	BG-2	UC-03	Э-1, Э-2	menu_items	TC-005…008
US-03	Must	BG-1	UC-06	Э-3	pickup_windows	TC-009…012
US-04	Must	BG-2, BG-3, BG-4	UC-05, UC-07	Э-4, Э-5	reservations, reservation_items, payments, orders, order_items	TC-013…017
US-05	Must	BG-1	UC-10	Э-6	orders, order_status_log	TC-018…020
US-06	Should	BG-1	UC-11	Telegram	students	TC-021…023
US-07	Must	BG-1	UC-12	Э-6	orders	TC-024…027
US-08	Should	BG-2, BG-3	UC-08, UC-09	Э-6	refunds, payments	TC-028…031
US-09	Could	BG-2	UC-04	—	orders, order_items	TC-032…034
US-10	Could	BG-2	UC-13	—	orders	TC-035…037
US-11	Must	BG-2	UC-01	—	students	TC-038…041
US-12	Must	BG-1	UC-14	Э-7	orders, pickup_windows	TC-042…045
US-13	Must	BG-1	UC-15	—	orders, order_status_log	TC-046…049
US-14	Must	BG-1, BG-3	UC-16	Э-7	orders	TC-050…053
US-15	Should	BG-4	UC-17	—	menu_items	TC-054…056
US-16	Must	BG-2, BG-4	UC-18	—	daily_menus, menu_items, dishes	TC-057…060
US-17	Should	BG-1	UC-19	—	pickup_windows	TC-061…063
US-18	Should	BG-3	UC-20	—	orders, order_items, payments	TC-064…067
US-19	Could	BG-1	UC-21	—	orders, order_status_log	TC-068…070
US-20	Won't	—	—	—	—	—

Все 19 историй первой версии имеют вариант использования и таблицы базы данных. 9 историй имеют экран прототипа. История US-06 работает через Telegram, поэтому у неё нет экрана. Истории без экрана — это истории администратора, часть историй сотрудника и вспомогательные истории студента; их экраны появятся на этапе 4. История US-20 (Won’t) не входит в первую версию, поэтому в строке нет связей.

Вывод. Трассировка не прерывается: от задачи BG через историю и вариант использования до таблицы базы данных и тест-кейса.

Выводы по главе 3

В главе 3 мы спроектировали сервис предзаказа обеда. Результаты:

- Процесс «как есть» имеет 5 проблем. Процесс «как будет» убирает их: 6 изменений, каждое связано с задачей BG-1…BG-4. Матрица RACI имеет одну роль A в каждой строке.
- 21 вариант использования покрывает 19 историй первой версии. Две диаграммы деятельности показывают альтернативные ветви. Диаграмма последовательности показывает, что данные карты не попадают в сервис.
- Схема базы данных имеет 13 таблиц, PK, FK и связи 1:N. Цена в позиции заказа хранится на момент оплаты; эта и ещё 5 копий обоснованы.
- Архитектура — модульный монолит на Django с 8 модулями, PostgreSQL 16 и Valkey, на сервере в Узбекистане. Для 6 интеграций есть меры против рисков.
- Прототип имеет 7 экранов и экран ошибки. Проверка на 5 участниках нашла 6 проблем, все исправлены. NFR-08 и NFR-09 выполнены на прототипе.
- Своя разработка на открытом стеке стоит около 102 млн сум за 3 года и окупается при 3 столовых и больше.

Что дальше. На этапе 4 мы пишем тест-кейсы ТС по диаграммам деятельности и рисуем экраны входа и администратора.

Приложение Г. Описание таблиц базы данных

Автор раздела: Алиев Салим (В)

Приложение описывает все поля 13 таблиц. SQL-скрипт схемы лежит в репозитории: /models/schema.sql.

Таблица Г.1 — staff (Сотрудники)

Поле	Тип	Ограничение	Описание
id	serial	PK	
name	varchar(60)	NOT NULL	Имя
phone	varchar(13)	UNIQUE	Телефон +998…
role	varchar(12)	NOT NULL, CHECK	«раздача» или «администратор»
password_hash	varchar(100)	NOT NULL	Хэш пароля (Argon2)
is_active	boolean	NOT NULL	Сотрудник работает

Таблица Г.2 — dishes (Блюда)

Поле	Тип	Ограничение	Описание
id	serial	PK	
name	varchar(80)	NOT NULL	Название блюда
portion_g	smallint	NOT NULL, CHECK > 0	Вес порции, г или мл
is_active	boolean	NOT NULL	Блюдо в продаже

Таблица Г.3 — daily_menus (Меню дня)

Поле	Тип	Ограничение	Описание
id	serial	PK	
menu_date	date	UNIQUE	Дата меню
published_at	timestamptz	NOT NULL	Время публикации
published_by	int	FK → staff.id	Кто опубликовал

Таблица Г.4 — menu_items (Позиции меню дня)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
daily_menu_id	int	FK → daily_menus.id	Меню дня
dish_id	int	FK → dishes.id	Блюдо; UNIQUE (daily_menu_id, dish_id)
price	integer	NOT NULL, CHECK > 0	Цена, сум
stock	smallint	NOT NULL, CHECK ≥ 0	Остаток, порций

Таблица Г.5 — students (Студенты)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
phone	varchar(13)	UNIQUE	Телефон +998…
name	varchar(60)	NOT NULL	Имя
telegram_chat_id	bigint	NULL	Для уведомлений; может быть пусто

Поле	Тип	Ограничение	Описание
created_at	timestamptz	NOT NULL	Время регистрации

Таблица Г.6 — pickup_windows (Окна получения)

Поле	Тип	Ограничение	Описание
id	serial	PK	
window_date	date	NOT NULL	Дата
break_name	varchar(30)	NOT NULL	Название перемены
starts_at	time	NOT NULL	Начало; UNIQUE (window_date, starts_at)
accept_until	time	NOT NULL	starts_at − 15 минут (FR-05)
order_limit	smallint	NOT NULL, DEFAULT 40	Лимит заказов (FR-21)

Таблица Г.7 — reservations (Резервы)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	Номер для платёжного сервиса
student_id	bigint	FK → students.id	Студент
window_id	int	FK → pickup_windows.id	Окно получения
total	integer	NOT NULL	Сумма, сум
status	varchar(10)	NOT NULL, CHECK	«активен», «оплачен», «истёк»
created_at	timestamptz	NOT NULL	Создан
expires_at	timestamptz	NOT NULL	created_at + 10 минут (FR-09)

Таблица Г.8 — reservation_items (Позиции резерва)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
reservation_id	bigint	FK → reservations.id	Резерв
menu_item_id	bigint	FK → menu_items.id	Позиция меню
qty	smallint	NOT NULL, CHECK 1…5	Количество (FR-03)
price	integer	NOT NULL	Цена в момент резерва

Таблица Г.9 — payments (Платежи)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
reservation_id	bigint	FK → reservations.id	Резерв
provider	varchar(8)	NOT NULL, CHECK	«payme», «click», «uzum»
provider_txn_id	varchar(64)	UNIQUE	ID транзакции платёжного сервиса
amount	integer	NOT NULL	Сумма, сум
status	varchar(10)	NOT NULL, CHECK	«создан», «оплачен», «отказ»
receipt_url	varchar(255)	NULL	Ссылка на фискальный чек
created_at	timestamptz	NOT NULL	Создан
confirmed_at	timestamptz	NULL	Подтверждён

Таблица Г.10 — orders (Заказы)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	Номер заказа
reservation_id	bigint	FK, UNIQUE	Резерв (1:1)
student_id	bigint	FK → students.id	Студент
window_id	int	FK → pickup_windows.id	Окно получения
order_date	date	NOT NULL	День заказа
pickup_code	char(4)	NOT NULL	Код; UNIQUE (order_date, pickup_code)
total	integer	NOT NULL	Сумма, сум
status	varchar(10)	NOT NULL, CHECK	«Принят», «Готовится», «Готов», «Выдан», «Отменён»
paid_at	timestamptz	NOT NULL	Время оплаты
arrived_at	timestamptz	NULL	«Я на месте» (FR-27)
issued_at	timestamptz	NULL	Время выдачи (FR-18)
issued_by	int	FK → staff.id, NULL	Кто выдал

Таблица Г.11 — order_items (Позиции заказа)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
order_id	bigint	FK → orders.id	Заказ
menu_item_id	bigint	FK → menu_items.id	Позиция меню
qty	smallint	NOT NULL, CHECK 1…5	Количество
price	integer	NOT NULL	Цена на момент оплаты

Таблица Г.12 — refunds (Возвраты)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
order_id	bigint	FK → orders.id	Заказ
payment_id	bigint	FK → payments.id	Платёж
amount	integer	NOT NULL	Сумма, сум
status	varchar(10)	NOT NULL, CHECK	«запрошен», «выполнен»
requested_at	timestamptz	NOT NULL	Запрошен (FR-14: ≤ 5 минут)
completed_at	timestamptz	NULL	Выполнен

Таблица Г.13 — order_status_log (Журнал статусов)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
order_id	bigint	FK → orders.id	Заказ
status	varchar(10)	NOT NULL	Новый статус
changed_at	timestamptz	NOT NULL	Время смены
changed_by	int	FK → staff.id, NULL	Кто сменил; пусто — сервис

Приложение Д. Сценарий проверки прототипа

Автор раздела: Алиев Салим (В)

Подготовка. Ведущий открывает прототип на смартфоне участника, наблюдатель готовит бланк и секундомер. Проверка одного участника длится 10–15 минут.

Текст для участника. «Мы проверяем приложение, а не вас. Ошибок здесь нет. Пожалуйста, говорите вслух, что вы видите и что хотите сделать. Мы не будем подсказывать.»

Задачи. Ведущий даёт задачи Т1–Т4 по одной (таблица 3.12). Время идёт от первого касания экрана до действия «Успех». Если участник молчит больше 30 секунд, ведущий спрашивает: «Что вы ищете?».

Вопросы после задач.

1) Что было самым трудным?
2) Что было непонятно на экране ошибки оплаты?
3) Оцените, насколько легко было сделать заказ: от 1 (очень трудно) до 7 (очень легко).
4) Стали бы вы заказывать обед так? Почему?

Таблица Д.1 — Бланк наблюдателя

Участник	Задача	Время, с	Успех (да/нет)	Где ошибся или остановился	Цитата
У_	Т1				
	Т2				
	Т3				
	Т4 (1)				
	Т4 (2)				

История изменений

Версия	Автор	Дата	Что изменилось
0.9	Команда 1	01.11.2026	Черновик моделей: BPMN «как есть» и «как будет», UML, схема БД, архитектура.
1.0	Команда 1	02.11.2026	Глава 3 полностью: разделы 3.1–3.7, прототип версии 2 после проверки на 5 участниках, раздел 3.6 о выборе платформы и ТСО, приложения Г и Д. Исправления по этапу 2 перечислены во введении.